Chapter 14 — Networking and Collective Communication

> *"A 10,000-GPU cluster is not 10,000 individual computers. It is a single parallel computer whose performance is determined not just by the GPUs but by the bandwidth and latency of the wires between them. A cluster that computes at 65% MFU with poor networking is a cluster running at 30% MFU."* — Distributed Systems Architecture Principle

---

Chapter Overview

When 1,024 GPUs synchronize gradients after a training step, terabytes of data must move between GPUs in under one second for the cluster to achieve acceptable MFU. The network is not secondary infrastructure — it is a first-class performance component. A well-configured InfiniBand fabric can achieve 85% scaling efficiency at 1,024 GPUs; a misconfigured one may achieve 40%.

This chapter builds the complete picture of AI cluster networking — from NVLink physics to RoCE congestion control to the NCCL environment variables that separate a 55% MFU cluster from a 38% MFU cluster.

Sections in this chapter:

- 14.1 The Complete Communication Stack — Five Levels
- 14.2 Intra-Node Fabric — NVLink, NVSwitch, and NVLS
- 14.3 InfiniBand — Architecture, SHARP, and GPUDirect RDMA
- 14.4 RoCEv2 — The Hyperscaler Alternative
- 14.5 Network Topology — Fat-Tree and Rail-Optimized Fabric
- 14.6 NCCL — Algorithms, Configuration, and Debugging
- 14.7 Compute-Communication Overlap [r2]
- 14.8 Scaling Efficiency — Measurement and Diagnosis [r2]
- 14.9 Key Takeaways and Review Questions

14.1 The Complete Communication Stack

Understanding where data moves — and which hardware moves it — is the prerequisite for every networking optimization decision.

```
AI CLUSTER COMMUNICATION HIERARCHY — FIVE LEVELS:

  Level  Scope              Technology          BW          Latency  Primary Use

──────────────────────────────────────────────────────────────────────────────
  L0     Within SM          SMEM / registers    ~100 TB/s   1 cycle  Kernel
compute
  L1     Within GPU         HBM via L2          H100: 3.35  400 cyc  Tensor ops
                                                  TB/s
  L2     Intra-node         NVLink 4.0 + NVSwitch 900 GB/s  <1 μs    Tensor
Parallel
         GPU-to-GPU                            (H100 8-GPU)
  L3     Inter-node         InfiniBand NDR      50 GB/s     ~1 μs    Data/Pipeline
         node-to-node       OR RoCEv2 400G      per port    ~3-5 μs  Parallel
  L4     Storage / cross-   Ethernet            12-50 GB/s  1-10 ms  Dataset, ckpt
         cluster                                per port

──────────────────────────────────────────────────────────────────────────────


  BANDWIDTH RATIOS (H100 cluster):
    L2 (NVLink) vs L3 (IB):    900 / 50  = 18× faster per GPU
    L3 (IB)     vs L4 (storage): 50 / 12 = ~4× faster for data access

  THE FUNDAMENTAL RULE:
    Place communication that requires LOW LATENCY on the LOWEST available level.
    TP AllReduce (every transformer layer, dozens per step) → MUST be L2 (NVLink)
    DP gradient sync (once per step, large message) → L3 (IB) acceptable
    PP activation passing (one hop per microbatch) → L3 (IB) one-hop acceptable

  NEVER run TP over InfiniBand:
    TP AllReduce: 2 per transformer layer × 80 layers = 160 AllReduces per step
    Over NVLink: 160 × 0.45ms = 72ms communication (18% of 400ms forward)
    Over IB NDR: 160 × 5.4ms = 864ms communication  (216% of forward — impossible)
    TP over IB: training is slower than single-GPU.
```

14.2 Intra-Node Fabric — NVLink, NVSwitch, and NVLS

14.2.1 NVLink Physical Architecture

```
NVLINK 4.0 — H100 SXM5 ARCHITECTURE: [SHIPPED]
```

```
  Per-GPU NVLink port count:  18 NVLink 4.0 lanes
  Per-GPU bandwidth:          900 GB/s bidirectional (all 8 GPUs combined)
  Per-link bandwidth:         50 GB/s bidirectional per NVLink lane

  NVSwitch (third-party fabric chip):
    6 NVSwitch chips per H100 DGX node
    Each NVSwitch: 64 NVLink ports
    Together: full all-to-all switching across all 8 GPUs
    Non-blocking: any GPU can send to any GPU at full 900 GB/s

  Topology: NON-BLOCKING ALL-TO-ALL within the 8-GPU node
    GPU 0 can send to GPU 7 at full bandwidth
    Simultaneously: GPU 1→5, GPU 2→6, GPU 3→4 all at full bandwidth
    No head-of-line blocking at any switch port

NVLINK 5.0 — B200 BLACKWELL: [SHIPPED]
  Per-GPU bandwidth: 1.8 TB/s bidirectional (2× NVLink 4.0)
  Per-link: 100 GB/s bidirectional
  GB200 NVL72: 72 GPUs in one rack with NVLink-speed interconnect
    → Effectively extends "single-node" NVLink domain to 72 GPUs
    → TP=72 possible without any InfiniBand crossing

AMD INFINITY FABRIC (MI300X): [SHIPPED]
  Intra-module bandwidth: ~896 GB/s (between chiplets within one MI300X)
  The MI300X is a single package with multiple GPU chiplets (XCDs)
  and HBM stacks — NVLink equivalent is internal to the package.
  Inter-module (MI300X to MI300X): standard IB or RoCE (same as H100 inter-node)
```

14.2.2 NVLS — In-Network AllReduce via NVSwitch

```
NVLS (NVLink SHARP) — H100 INTRA-NODE IN-NETWORK AGGREGATION:

  Problem being solved:
    Standard intra-node AllReduce (Ring algorithm):
      GPU 0 sends partial sum to GPU 1 → GPU 1 computes, sends to GPU 2 ...
      GPUs spend SM cycles on summation and transfer management.
      HBM bandwidth consumed by passing partial sums between GPUs.

  NVLS solution:
    NVSwitch hardware contains dedicated compute units (aggregation hardware).
    GPUs write gradients to NVSwitch-visible memory.
    NVSwitch aggregates (sums) all inputs in hardware.
    NVSwitch multicasts result back to all GPUs simultaneously.
```

```
    GPU SMs are NEVER involved in the summation computation.
    GPUs only write once and read once (not K-1 sends in ring).

  PERFORMANCE IMPACT (8-GPU AllReduce, large message):
    Ring AllReduce (no NVLS):   saturates NVLink at ~900 GB/s, high SM cost
    NVLS AllReduce:             ~2× throughput (NVSwitch compute + multicast)
                                 SM cycles freed for actual model computation

  ENABLING NVLS:
    Automatic on H100/H200 in NCCL ≥ 2.16. No configuration needed.
    Verify: NCCL_DEBUG=INFO python train.py 2>&1 | grep NVLS
    Expected: "NCCL INFO Using NVLS protocol"

  NVLS LIMITATION:
    Only works intra-node (NVSwitch-connected GPUs).
    Inter-node AllReduce: standard Ring/Tree algorithm over IB.
    Hybrid (common in practice): NVLS intra-node + Ring inter-node.
```

14.3 InfiniBand — Architecture, SHARP, and GPUDirect RDMA

14.3.1 InfiniBand Stack

```
INFINIBAND ARCHITECTURE (NDR 400G, current production standard):

  PER-NODE CONFIGURATION (H100 DGX, 8 GPUs per node):
    8 × ConnectX-7 NICs (one per GPU)
    Each NIC: 400 Gbps = 50 GB/s bidirectional
    Per-node total: 8 × 50 GB/s = 400 GB/s bidirectional

  HARDWARE SPECIFICATIONS (NDR 400G): [SHIPPED]
    Link rate:        400 Gbps per port (= 50 GB/s)
    Effective rate:   ~360-380 Gbps (accounting for 8/10 encoding overhead)
    Latency:          ~1 μs end-to-end (NIC to NIC across one switch hop)
    RDMA:             YES — GPU HBM → NIC DMA buffer directly (no CPU)
    Switch radix:     64 ports (Quantum2), 128 ports (Quantum-X800)
    Switch aggregate: 40 Tbps (Quantum2), 80 Tbps (Quantum-X800)

  QUANTIZING INTERCONNECT HARDWARE (QDR → HDR → NDR → XDR):
    QDR (40G):   Legacy, not suitable for modern AI
    EDR (100G):  V100-era, acceptable for 8-GPU nodes
```

```
    HDR (200G):  A100-era standard, deployed widely
    NDR (400G):  H100-era current standard; DGX SuperPOD default
    XDR (800G):  Quantum-X800, available 2025; required for B200 clusters

  NIC SELECTION AND PLACEMENT:
    CPU NUMA affinity: each NIC should be on same PCIe root complex as its GPU.
    PCIe topology mapping: nvidia-smi topo -m
      Shows: GPU-to-GPU and GPU-to-NIC topology with PCIe distances.
    Target: each GPU's NIC shows "PIX" (PCIe switched, same root) or "NV#"
(NVLink).
    Avoid: "SYS" (different NUMA node) — adds NUMA crossing latency to every RDMA.
```

14.3.2 IB SHARP — Inter-Node In-Network AllReduce

```
IB SHARP (Scalable Hierarchical Aggregation and Reduction Protocol):

  CONCEPT:
    Like NVLS but for InfiniBand switches.
    Aggregation compute units inside Quantum2 switches perform AllReduce.
    Gradients are aggregated as they flow THROUGH the switch fabric.
    GPUs send once; switch does the math; GPUs receive once.

  WITHOUT SHARP:
    Standard Ring AllReduce traffic (N GPUs, message size M):
      Total traffic = 2 × (N-1)/N × M × N  (each GPU sends and receives full M)
      For N=1024, M=140 GB: each GPU transfers ~280 GB per AllReduce

  WITH SHARP:
    Traffic reduction: ~50% (switch aggregates in-flight; fewer round-trips)
    Latency reduction: ~30-40% (fewer switch hops via aggregation tree)
    SM cycles freed:    GPU SMs do no summation work
    Switch QPS consumed: aggregation uses switch ASIC compute resources

  REQUIREMENTS:
    InfiniBand ONLY (not available on RoCE — fundamental limitation).
    SHARP-capable switches (Quantum2 or later).
    sharpd daemon running on all nodes.
    NCCL ≥ 2.12 with SHARP support compiled in.

  CONFIGURATION:
    export NCCL_SHARP_ENABLE=1
    export SHARP_COLL_LOCK_ON_COMM_INIT=1   # Pre-allocate SHARP resources at init
    export NCCL_SHARP_GROUP_SIZE_THRESH=16  # Use SHARP only for groups ≥ 16 ranks
```

```
  WHEN SHARP IS NOT WORTH ENABLING:
    Small clusters (< 16 nodes): Ring AllReduce is competitive; SHARP overhead.
    Small messages (< 64 KB): latency dominates; SHARP setup cost not amortized.
    MoE All-to-All operations: SHARP is designed for AllReduce, not All-to-All.
```

14.3.3 GPUDirect RDMA — The Zero-CPU Data Path

```
GPUDIRECT RDMA:

  STANDARD PATH (without GDR):
    Send: GPU HBM → PCIe → CPU stage buffer → PCIe → NIC DMA → Network
    Recv: Network → NIC DMA → CPU stage buffer → PCIe → GPU HBM
    Latency overhead: ~2-3 μs (two CPU crossing round-trips)
    CPU utilization: 16+ CPU cores per node (8 NICs × 2 proxy threads each)
    Bandwidth cap: Limited by CPU PCIe transfer rate

  GPUDIRECT RDMA PATH:
    Send: GPU HBM → NIC reads via PCIe P2P BAR access → Network
    Recv: Network → NIC writes directly to GPU HBM via PCIe P2P
    Latency overhead: ~0.3-0.5 μs (PCIe P2P, no CPU involvement)
    CPU utilization: 2-4 cores (metadata only; no data movement)
    Bandwidth: Full NIC line rate (380 Gbps effective)

  CPU SAVINGS AT SCALE:
    Without GDR: 16 proxy threads per node × 1,024 nodes = 16,384 CPU cores
                 dedicated to gradient copying. On a 1024-node H100 cluster:
                 16,384 cores is roughly the CPU count of 1,024 server CPUs!
    With GDR:    ~4,096 cores for metadata → 12,288 CPU cores freed.
    Performance: 10-20% improvement in communication-bound step times.

  VERIFYING GPUDirect RDMA:
    # Check P2P access capability:
    nvidia-smi topo -p2p r
    # OK | PIX | NV# = GDR capable; HOST = falls back to CPU path

    # Confirm NCCL using GDR at runtime:
    NCCL_DEBUG=INFO python train.py 2>&1 | grep "GDR\|GPUDirect"
    # Expected: "NCCL INFO GDR enabled"

    # Benchmark direct P2P bandwidth (GPU→NIC):
    ib_write_bw -d mlx5_0 -x 3 --report_gbits   # On receiver
    ib_write_bw -d mlx5_0 -x 3 <server_ip>       # On sender
```

```
    # Target: ≥ 380 Gbps (95% of 400G line rate)
    # Alert if: < 340 Gbps → possible GDR fallback or link degradation
```

14.4 RoCEv2 — The Hyperscaler Alternative

14.4.1 Why RoCE Instead of InfiniBand

```
ROCEVS vs INFINIBAND — THE STRATEGIC DECISION:

  InfiniBand (NVIDIA/Mellanox path):
    Dedicated network fabric, proprietary switches and NICs
    Best latency: ~1 μs end-to-end
    IB SHARP: in-network AllReduce (major throughput advantage)
    Higher cost: ~2-3× more expensive than equivalent Ethernet
    Vendor lock-in: NVIDIA Quantum switches + Mellanox NICs
    Deployed by: most NVIDIA-centric cloud providers, research clusters

  RoCEv2 (Meta/hyperscaler path):
    RDMA over standard Ethernet switches (Arista, Broadcom, etc.)
    Latency: ~3-5 μs (3-5× worse than IB for end-to-end)
    NO IB SHARP: requires software AllReduce algorithms
    Cost: standard Ethernet switches significantly cheaper
    Flexibility: mix with regular data center Ethernet fabric
    Deployed by: Meta (Llama-3 training), Microsoft, Amazon

  META'S DECISION (published SIGCOMM 2024):
    Meta trained Llama-3 (405B) on 24,576-GPU RoCEv2 clusters.
    Hardware: Arista 7800 + Wedge400 + Minipack2 OCP switches at 400 Gbps.
    Reason: cost, vendor diversity, Ethernet ecosystem tooling.
    Challenge: standard RoCE congests at 400G+ speeds → required custom
    congestion control (CTS protocol, see §14.4.3).
    Result: competitive MFU vs InfiniBand equivalent after CTS co-design.
```

14.4.2 Making Ethernet Lossless — PFC

```
THE LOSSLESSNESS REQUIREMENT:

  RDMA assumes lossless delivery: if a packet is dropped, RDMA semantics
  break (the remote write partially completed → memory corruption).
  Standard TCP handles packet loss with retransmission (acceptable for
  storage/web). RDMA cannot: retransmission disrupts the DMA mapping.
```

```
  Ethernet is inherently lossy (congestion → tail drops).
  Solution: Priority Flow Control (PFC) — IEEE 802.1Qbb.

PFC MECHANISM:
  When a switch port's buffer fills to a threshold:
    Switch sends PAUSE frame to upstream sender.
    Sender stops transmitting on that priority queue.
    Buffer drains. Switch sends UNPAUSE. Sender resumes.

  Effect: No packet drops for RDMA traffic (priority class 3 typically).
  No drops = RDMA semantics preserved. Lossless delivery guaranteed.

  PFC CONFIGURATION:
    Enable PFC on all RDMA-carrying switch ports (all trunk links).
    Set PFC priority class: typically class 3 for RDMA (DSCP=26).

    # On Mellanox NIC (mlnx_qos tool):
    sudo mlnx_qos -i eth0 --pfc 0,0,0,1,0,0,0,0  # Enable PFC on priority 3

    # On Arista switch (RoCE config):
    interface Ethernet1/1
      priority-flow-control on
      priority-flow-control priority 3 no-drop

PFC FAILURE MODE — HEAD-OF-LINE BLOCKING:
  If traffic from source A and source B shares a switch port:
    A is congested → PFC pause sent to BOTH A and B.
    B was not congested, but is paused anyway.
    B's throughput collapses due to A's congestion → collateral damage.

  This "PFC storm" can propagate backwards through the fabric.

  Mitigation: Separate lossless (RDMA, priority 3) from lossy (IP, priority 0)
  traffic using strict PFC priority separation. QoS policy prevents PFC from
  propagating onto non-RDMA traffic classes.
```

14.4.3 Congestion Control Beyond PFC — DCQCN and Meta's CTS

```
WHY PFC ALONE IS INSUFFICIENT AT 400G:

  PFC prevents drops but does NOT prevent congestion buildup.
  At 400G, even a 1μs congestion event fills large switch buffers:
```

```
    1μs × 400 Gbps = 50 KB of in-flight data per link
    Modern switch buffers: 32-512 MB

  Without active congestion control:
    Traffic bursts → switch buffer fills → PFC pause → fabric-wide pause storm
    At 24,576 GPUs: one congestion event cascades across thousands of flows.

DCQCN (Data Center Quantized Congestion Notification):
  Standard congestion control for RoCE (RFC 8257, IEEE 802.1Qau ECN).

  Mechanism:
    1. Congested switch sets ECN bit in packet header (Explicit Congestion
Notification).
    2. Receiver notices ECN bit → sends CNP (Congestion Notification Packet) to
sender.
    3. Sender receives CNP → reduces injection rate (multiplicative decrease).
    4. Sender slowly increases rate after no CNPs received (additive increase).

  Behavior: TCP AIMD (Additive Increase, Multiplicative Decrease) for RDMA.
  Works well for moderate traffic but struggles at 400G with GPU bursty traffic.

META'S CTS (Clear to Send) — RECEIVER-DRIVEN ADMISSION:
  Meta co-designed NCCL and network transport for Llama-3 training.

  Problem with sender-driven DCQCN at GPU scale:
    GPUs inject data in synchronized bursts (every AllReduce start).
    All N GPUs inject simultaneously → incast congestion → PFC storm.
    DCQCN reacts after congestion; incast congestion is front-loaded.

  CTS solution: send before congestion occurs.
    Step 1: Sender GPU copies gradient data to HBM staging buffer.
    Step 2: Sender's NIC waits for CTS (permission-to-send) from receiver.
    Step 3: Receiver grants CTS only when it has buffer space ready.
    Step 4: On CTS receipt: RDMA write proceeds immediately.

  Effect:
    No incast: senders don't transmit until receivers are ready.
    Switch buffer buildup reduced by 10× vs DCQCN.
    Meta's benchmark: Llama-3 → Llama-4 training, same hardware:
      CTS reduced NCCL stalls by >50% on their RoCE fabric.

  Availability: Part of Meta's open-source NCCL fork contributions.
```

14.5 Network Topology — Fat-Tree and Rail-Optimized Fabric

14.5.1 Two-Tier Fat-Tree

```
FAT-TREE TOPOLOGY (standard AI cluster fabric):

  TWO-TIER DESIGN:
    Tier 1 (Leaf):  One switch per rack or group of racks
                    Ports down: server NICs | Ports up: spine uplinks
    Tier 2 (Spine): Spine switches interconnect all leaf switches

  CONFIGURATION EXAMPLE (1,024 GPUs = 128 nodes × 8 GPUs/node):
    Each node: 8 GPUs × 1 NIC = 8 × 400 Gbps = 3.2 Tbps per node
    Nodes per leaf: 4 (using Quantum2 with 64 ports: 32 down + 32 up)
    Leaf switches needed: 128 nodes / 4 nodes/leaf = 32 leaves
    Spine switches: 32 uplinks per leaf = 32 spine ports needed per leaf
    With Quantum2 (32 uplink ports): 1 spine connects all leaves = 1:1 non-
blocking

  OVERSUBSCRIPTION RATIO:
    Oversubscription = total downlink bandwidth / total uplink bandwidth
    1:1 (non-blocking): every server gets full bandwidth to every other server
                        Required for all-to-all heavy workloads (MoE training)
    2:1: half bandwidth at spine → hotspots under all-to-all traffic

    AI training requirement: 1:1 or 2:1 at most
    DGX SuperPOD design: 1:1 non-blocking fat-tree
    Cost tradeoff: 1:1 requires 2× the spine switch ports vs 2:1
```

14.5.2 Rail-Optimized Topology

```
RAIL-OPTIMIZED FABRIC — THE CRITICAL TOPOLOGY OPTIMIZATION:

CONCEPT:
  In a standard fat-tree, GPUs on the same node share NICs in any configuration.
  In a rail-optimized fabric: GPU-i on EVERY node connects to the SAME leaf
switch.

  Example (8 rails, one NIC per GPU):
    GPU 0 on node 0 → Rail 0 leaf switch
    GPU 0 on node 1 → Rail 0 leaf switch
    GPU 0 on node 7 → Rail 0 leaf switch
```

```
    ...
    GPU 1 on all nodes → Rail 1 leaf switch
    ...
    GPU 7 on all nodes → Rail 7 leaf switch

WHY THIS MATTERS FOR ALLREDUCE:

  Ring AllReduce NCCL pattern (N-GPU ring, each GPU talks to next):
    Step 1: GPU-i on node A sends chunk to GPU-i on node B
    Step 2: GPU-i on node B sends chunk to GPU-i on node C
    ...
    All communication is GPU-i → GPU-i (same GPU index, different nodes)

    WITH RAIL-OPTIMIZED FABRIC:
      GPU-i traffic always flows through Rail-i leaf switch.
      Zero cross-rail traffic for AllReduce.
      Each rail carries exactly 1/N_rails of total AllReduce volume.
      Maximum bandwidth utilization, no hotspots.

    WITHOUT RAIL-OPTIMIZED FABRIC:
      GPU-0 on node A may communicate to GPU-0 on node B via Rail-2
      (if they happen to share a different NIC assignment).
      Cross-rail traffic → competes on spine uplinks with other GPUs.
      Spine contention → 30-50% AllReduce throughput reduction.

  PERFORMANCE IMPACT:
    With rail-optimized: NCCL AllReduce at 85-90% of theoretical NIC bandwidth.
    Without rail-optimized: NCCL AllReduce at 50-60% (spine hotspot).
    That 30-40% gap represents ~10-20% MFU difference for communication-heavy
    training jobs.

IMPLEMENTING RAIL-OPTIMIZED IN NCCL:
  1. Physical wiring: ensure GPU-i on every node connects to same leaf switch.
  2. NIC ordering: NCCL detects topology via NCCL_TOPO_DUMP_FILE.
  3. NCCL_IB_HCA: explicitly pin NCCL to use NIC-i for GPU-i:
     export
NCCL_IB_HCA=mlx5_0:1,mlx5_1:1,mlx5_2:1,mlx5_3:1,mlx5_4:1,mlx5_5:1,mlx5_6:1,mlx5_7:1
     # mlx5_N corresponds to NIC for GPU N — critical ordering!

  4. Verify topology detection:
     NCCL_TOPO_DUMP_FILE=/tmp/nccl_topo.xml python -c \
       "import torch.distributed; torch.distributed.init_process_group('nccl')"
```

```
    cat /tmp/nccl_topo.xml  # Shows GPU-NIC-Switch topology
    # Look for: Gpu→Net connections following rail pattern
```

14.6 NCCL — Algorithms, Configuration, and Debugging

14.6.1 NCCL Collective Operations

```
NCCL COLLECTIVE REFERENCE:

  Operation        Input           Output          Use in AI         Algorithm

──────────────────────────────────────────────────────────────────────────────

  AllReduce        All GPUs: Xi    All GPUs: ΣXi   DP gradient sync  Ring, Tree,
NVLS
  ReduceScatter    All GPUs: Xi    GPU-i: shard(ΣXi) FSDP/ZeRO       Ring
  AllGather        GPU-i: Xi       All GPUs: X0..N FSDP param        Ring
  Broadcast        GPU-0: X        All GPUs: X     Model init        Tree
  Reduce           All GPUs: Xi    GPU-0: ΣXi      Monitoring sum    Tree
  All-to-All       GPU-i: {Xij}   GPU-j: {Xji}    MoE expert        Pairwise
                                                     routing
  Point-to-Point   One → One       —               PP activation     Direct RDMA

──────────────────────────────────────────────────────────────────────────────


ALLREDUCE ALGORITHM SELECTION (auto in NCCL based on message size):

  Message < 64 KB:   TREE algorithm (latency-optimal)
    Binary broadcast tree structure. Fewer hops for small messages.
    Time ∝ O(log N) hops × latency (not bandwidth-limited)

  Message > 1 MB:    RING algorithm (bandwidth-optimal)
    All GPUs form ring; pass data sequentially.
    Time ∝ 2 × (N-1)/N × msg_size / bandwidth
    At large N: approaches 2 × msg_size / bandwidth (near-linear)

  Intra-node (NVLink): NVLS (in-network aggregation)
    NVSwitch hardware performs aggregation.
    ~2× faster than Ring for intra-node at any message size.
    Auto-selected when NVLink topology detected.

  Override:
```

```
export NCCL_ALGO=RING    # Force Ring for all operations
export NCCL_ALGO=TREE    # Force Tree
# Only override if debugging; NCCL auto-selection is well-tuned
```

14.6.2 Critical NCCL Environment Variables

```bash
```

```
# ═══════════════════════════════════════════════════════════════════
# NCCL PRODUCTION CONFIGURATION — H100 CLUSTER WITH IB NDR 400G
# ═══════════════════════════════════════════════════════════════════

# ── TOPOLOGY AND NIC SELECTION ──────────────────────────────────
# Pin NCCL to specific IB HCAs (one per GPU, in GPU order)
export NCCL_IB_HCA=mlx5_0:1,mlx5_1:1,mlx5_2:1,mlx5_3:1,mlx5_4:1,mlx5_5:1,mlx5_6:1,ml
# CRITICAL: Order must match GPU PCI topology. Verify with nvidia-smi topo -m.
# Wrong ordering = cross-NUMA traffic = 30% performance degradation

# ── TRANSPORT SETTINGS ──────────────────────────────────────────
export NCCL_IB_QPS_PER_CONNECTION=4    # Queue pairs per connection (default: 1)
                                       # 4 QPs → better NIC ASIC utilization
                                       # Especially important at 400G line rate
export NCCL_IB_GID_INDEX=3            # GID index for RoCE/IB (match your subnet con
export NCCL_IB_TIMEOUT=22             # IB timeout (22 = ~4 seconds). Increase for d
export NCCL_NET_GDR_LEVEL=SYS         # Enable GPUDirect RDMA (SYS = maximum scope)

# ── BUFFER AND PERFORMANCE ──────────────────────────────────────
export NCCL_BUFFSIZE=8388608          # Intermediate buffer: 8 MB (default: 4 MB)
                                       # Larger = better BW for large gradients
                                       # Too large = OOM risk on small-memory GPUs
export NCCL_MIN_NCHANNELS=4          # Minimum communication channels per peer
                                       # Increase to 4-8 for 400G NICs (more paral
export NCCL_MAX_NCHANNELS=16         # Cap channels (too many = NIC resource exhaust

# ── IN-NETWORK AGGREGATION ──────────────────────────────────────
export NCCL_NVLS_ENABLE=1             # Enable NVLS (default on H100+, set explicitl
export NCCL_SHARP_ENABLE=1           # Enable IB SHARP (requires Quantum2+ switches)
export SHARP_COLL_LOCK_ON_COMM_INIT=1  # Pre-allocate SHARP resources at job start
export NCCL_SHARP_GROUP_SIZE_THRESH=16  # Only use SHARP for ≥ 16-rank groups

# ── PROTOCOL ────────────────────────────────────────────────────
export NCCL_PROTO=LL128               # Low-latency 128-byte aligned protocol
                                       # Alternatives: Simple (large BW), LL (smal
                                       # LL128: best balance for production traini

# ── TIMEOUT AND RELIABILITY ─────────────────────────────────────
export NCCL_TIMEOUT=1800              # 30 minutes — increase for large clusters
                                       # Default (300s) triggers false failures at
export NCCL_SOCKET_NTHREADS=4        # CPU threads for socket-based fallback
export NCCL_PROXY_LAUNCH_MODE=fork   # fork | pthread. fork = safer with CUDA contex
```

```
# ═══════════════════════════════════════════════════════════════════════
# DEBUGGING — USE ONLY WHEN DIAGNOSING ISSUES (high overhead)
# ═══════════════════════════════════════════════════════════════════════
# export NCCL_DEBUG=INFO              # Basic info (ring topology, algorithm chosen)
# export NCCL_DEBUG=WARN              # Warnings and errors only
# export NCCL_DEBUG_SUBSYS=ALL        # With DEBUG=INFO: verbose per-operation loggi
# export NCCL_P2P_DISABLE=1          # Disable P2P (forces host fallback — for testi
```

14.6.3 NCCL-Tests — Cluster Health Verification

```bash
```

```
# ─── nccl-tests: The Essential Cluster Health Benchmark ──────────────────────

# Build nccl-tests (one-time):
git clone https://github.com/NVIDIA/nccl-tests
cd nccl-tests && make MPI=1 MPI_HOME=/usr/local/mpi CUDA_HOME=/usr/local/cuda

# ─── AllReduce bandwidth sweep (most important test) ────────────────────────
mpirun -np 8 \                      # 8 processes for 8-GPU node
  --hostfile hostfile \             # List of nodes (one per line)
  ./build/all_reduce_perf \
  -b 1M -e 4G -f 2 \               # Sweep: 1 MB to 4 GB, doubling each step
  -g 1 \                            # 1 GPU per process
  -c 1                              # Check correctness

# Expected H100 × 8 (NVLink, intra-node):
#   Message 256MB: busbw ≥ 160 GB/s
#   Message 1GB:   busbw ≥ 180 GB/s
#   Message 4GB:   busbw ≥ 185 GB/s
#   Alert threshold: busbw < 140 GB/s → NVLink degradation

# Expected H100 × 64 (InfiniBand NDR 400G, 8-node):
#   Message 256MB: busbw ≥ 40 GB/s
#   Message 1GB:   busbw ≥ 45 GB/s
#   Message 4GB:   busbw ≥ 48 GB/s
#   Alert threshold: busbw < 35 GB/s → IB link degradation

# ─── Point-to-point bandwidth (diagnose specific pair) ──────────────────────
mpirun -np 2 --hostfile pair_hosts ./build/sendrecv_perf -b 1M -e 4G -f 2
# Alert: < 47 GB/s (94% of 50 GB/s NDR link) → physical link issue

# ─── Scaling efficiency test (all-to-all verification) ──────────────────────
for n in 8 64 128 256 512 1024; do
  mpirun -np $n ./build/all_reduce_perf -b 1G -e 1G -f 2 -g 1 \
    | grep "1073741824" | awk -v n=$n '{print n, $9}'  # Extract busbw at 1GB
done
# Expected: busbw should NOT decrease significantly as N grows
# If busbw drops > 20% from 64→128 GPUs: topology or routing issue
```

14.6.4 NCCL Tuning Impact — The 10-30% Rule

```
NCCL TUNING YIELDS 10-30% COMMUNICATION IMPROVEMENT:

  Baseline (default NCCL):
```

```
    H100 × 8 AllReduce busbw at 1 GB:    ~120 GB/s
    H100 × 64 AllReduce busbw at 1 GB:   ~28 GB/s

  After tuning (production config above):
    H100 × 8 AllReduce busbw at 1 GB:    ~175 GB/s  (+46%)
    H100 × 64 AllReduce busbw at 1 GB:   ~43 GB/s   (+54%)

  PRIMARY GAINS:
    NCCL_IB_HCA pin (rail alignment):   +20-30% (biggest single change)
    NCCL_IB_QPS_PER_CONNECTION=4:       +10-15% (NIC ASIC utilization)
    NCCL_BUFFSIZE 8M:                   +5-10% (fewer kernel calls)
    NVLS/SHARP enable:                  +15-20% (in-network aggregation)
    NCCL_MIN_NCHANNELS=4:               +5-8%  (more parallel streams)
    TOTAL COMBINED:                     20-40% improvement

  TRANSLATION TO MFU (30% communication → 15% step time):
    If AllReduce = 20% of step time and communication improves 30%:
    Step time reduction = 20% × 30% = 6% improvement
    MFU: 38% → 40.3% (6% relative improvement)

    At 10,000 GPUs: 2.3% MFU × 10K GPUs = 230 GPU-equivalent hours/day saved
    At $1.66/GPU-hr: $382/day or $139,000/year from NCCL tuning alone.
```

14.7 Compute-Communication Overlap [r2]

14.7.1 The Overlap Imperative

```
WHY OVERLAP IS THE PRIMARY MFU LEVER FOR LARGE CLUSTERS:

At 512-GPU scale (typical enterprise training):
  GPU compute per step:       1,200 ms
  AllReduce (unoptimized):      800 ms
  Total without overlap:      2,000 ms  → MFU = 1200/2000 = 60%
  Total with perfect overlap: 1,200 ms  → MFU = 1200/1200 = 100%

  Perfect overlap: the entire AllReduce is hidden behind compute.
  Even 70% overlap: 1200 + 0.30×800 = 1,440 ms → MFU = 83%

  Going from 0% to 70% overlap: MFU doubles (60% → 83%).
  This is the most impactful training optimization at this scale.
```

14.7.2 Overlap Mechanisms and Configuration

```
OVERLAP MECHANISM 1 — DDP BUCKET OVERLAP (Data Parallel):

  PyTorch DDP fires AllReduce as soon as each gradient bucket fills.
  Subsequent backward layers compute while prior layers' gradients sync.

  Configuration:
    model = DDP(model, bucket_cap_mb=25,                # 25 MB buckets
                find_unused_parameters=False,            # Disable unused-param
search
                gradient_as_bucket_view=True)            # Zero-copy gradient
staging

  How it works:
    backward layer L computes → bucket fills → AllReduce fires on NCCL stream
    backward layer L-1 computes on compute stream (simultaneous)
    AllReduce for L completes → bucket L starts optimizer step

  Verification in nsys timeline:
    HEALTHY: NCCL allreduce bars appear BELOW backward compute bars (overlapped)
    BROKEN:  NCCL allreduce bars appear AFTER backward (sequential = no overlap)

OVERLAP MECHANISM 2 — FSDP PREFETCH:

  FSDP AllGather: prefetch next layer's full weights while computing current
layer.
  FSDP ReduceScatter: scatter current layer's gradients while backward advances.

  Configuration:
    from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
    from torch.distributed.fsdp import BackwardPrefetch

    model = FSDP(
        model,
        backward_prefetch=BackwardPrefetch.BACKWARD_PRE,  # Prefetch before
backward
        forward_prefetch=True,                             # Prefetch during
forward
        limit_all_gathers=True,                            # Memory safety limiter
    )

OVERLAP MECHANISM 3 — ASYNC GRADIENT ACCUMULATION:
```

```
  Gradient accumulation (K microbatches before gradient sync):
  WRONG: sync after EACH microbatch (K-1 unnecessary AllReduces)

    # BUG: DDP will AllReduce gradients every microbatch
    for batch in loader:
        loss = model(batch); loss.backward()  # Triggers AllReduce each time!

  CORRECT: suppress sync for accumulation steps, only sync on last

    for i, batch in enumerate(loader):
        ctx = model.no_sync() if i % K != K-1 else contextlib.nullcontext()
        with ctx:
            loss = model(batch)
            loss.backward()

    # Only the K-th backward triggers AllReduce

OVERLAP FAILURE MODES:
  1. Gradient norm clipping before AllReduce:
     torch.nn.utils.clip_grad_norm_(parameters, max_norm=1.0)
     This requires ALL gradients to be present → waits for full backward → blocks
     NCCL launch until backward completes. Fix: clip_grad after AllReduce (in FSDP
     mode) or use pre-computed norm estimate.

  2. Loss.backward() synchronization point:
     If loss is computed with any torch.cuda.synchronize() → serializes pipeline.
     Check: any custom loss functions with CPU-side reductions.

  3. Model has unused parameters:
     DDP stalls when some parameter has no gradient (no grad → no AllReduce
trigger).
     Fix: find_unused_parameters=True (overhead) or remove unused branches.
```

14.8 Scaling Efficiency — Measurement and Diagnosis [r2]

14.8.1 Scaling Efficiency Definition and Targets

```
SCALING EFFICIENCY:

  Definition:
```

```
    scaling_efficiency(N) = throughput(N GPUs) / (N × throughput(1 GPU))

  Ideal: 100% (N GPUs give exactly N× throughput of 1 GPU)
  Reality: communication overhead prevents 100% at any scale > 1

EXPECTED SCALING EFFICIENCY BY CLUSTER SIZE (H100, IB NDR 400G):

  GPU Count   Communication     Expected Eff.   Alert Threshold
  ──────────────────────────────────────────────────────────────
  8           NVLink only        92-97%          < 88%
  64          1-hop IB          85-92%          < 80%
  256         2-hop IB          80-88%          < 72%
  1,024       2-3 hop IB        74-85%          < 65%
  4,096       3-hop IB          68-80%          < 58%
  ──────────────────────────────────────────────────────────────

  UNDER 80% AT 64 GPUS: almost always a configuration problem, not physics.
  Common causes: NIC affinity wrong, overlap not configured, IB link degraded.

MEASURING SCALING EFFICIENCY:

  # Single-GPU baseline (establish reference):
  CUDA_VISIBLE_DEVICES=0 python train.py --profile 100 steps
  # Note: tokens_per_second (global)

  # Multi-GPU measurement (same 100 steps):
  torchrun --nproc_per_node=8 --nnodes=N train.py --profile 100 steps
  # Note: tokens_per_second (global)

  # Calculate:
  efficiency = (multi_tps / N) / single_tps

  # Run with nccl-tests to get network baseline:
  mpirun -np N ./all_reduce_perf -b 1G -e 1G -f 2 -g 1
  # Compare to single-node busbw to isolate network degradation
```

14.8.2 Scaling Efficiency Diagnosis

```
SCALING EFFICIENCY DIAGNOSIS DECISION TREE:

SYMPTOM: Scaling efficiency at 64-GPU < 80%

STEP 1: Is NCCL bandwidth degraded vs single-node baseline?
```

```
  Run nccl-tests on 64-GPU:
    busbw < 35 GB/s (IB NDR): network problem
      → Check ibstatus on all nodes for link speed / physical state
      → Check NCCL_IB_HCA is pinned correctly (rail alignment)
      → Check for failing switch ports (ibdiag, ibnetdiscover)
    busbw ≥ 35 GB/s: communication bandwidth not the bottleneck → Step 2

STEP 2: Is AllReduce overlapped with backward?
  Run nsys profile --trace=cuda,nccl python train.py
  Check timeline: NCCL operations overlapping backward compute?
    NOT overlapping: overlap configuration problem → fix bucket config /
backward_prefetch
    OVERLAPPING: overlap is working → Step 3

STEP 3: Is pipeline parallelism bubble fraction high?
  If using PP: calculate bubble = (PP-1)/(PP-1+M)
    If > 10%: increase M (microbatches) to reduce bubble → Step 4
    If < 10%: PP is not the bottleneck → Step 4

STEP 4: Is data loading a bottleneck?
  Check: nsys timeline for gaps before each forward pass start
    Gaps > 5ms: data loading stall → increase DataLoader num_workers
    No gaps: data pipeline is not bottleneck → Step 5

STEP 5: Is there a straggler GPU?
  Per-node step time variance (from training log):
    One node consistently 10%+ slower: hardware degradation (thermal, ECC, IB
link)
    → Run GPU health check on outlier node; replace if degraded

REFERENCE DIAGNOSIS TABLE:

  Symptom                        Root Cause               Fix
  ──────────────────────────────────────────────────────────────────────────
  NCCL busbw < 35 GB/s           IB link degraded         ibstatus + NCCL_IB_HCA
  AllReduce sequential in nsys   No overlap configured
BackwardPrefetch.BACKWARD_PRE
  busbw good, efficiency low     Wrong NCCL_IB_HCA order  Realign to rail topology
  Periodic latency spikes        NCCL timeout too short   NCCL_TIMEOUT=1800
  One node 20% slower            Straggler hardware       Replace/drain node
  ──────────────────────────────────────────────────────────────────────────
```

14.8.3 Parallelism-to-Fabric Mapping

```
4D PARALLELISM → PHYSICAL FABRIC MAPPING (Production Rule):

  Parallelism     Physical Fabric          Reason
  ──────────────────────────────────────────────────────────────────────

  TP              NVLink (same node)        Latency-critical: 160 AllReduces/step
                                            IB would add 864ms vs 72ms on NVLink

  PP              IB, adjacent nodes        One hop only; point-to-point RDMA
                  (same rack preferred)     Activation volume manageable over IB

  DP/FSDP         IB, any topology          Once per step, large message
                                            Overlapped with compute → latency OK

  CP (sequence)   NVLink (preferred)        All-to-All for attention; ideally same
node
                  IB acceptable if > 8 CP   For CP > 8: must cross node boundary

  EP (MoE)        NVLink (EP ≤ 8)          All-to-All needs bandwidth
                  IB (EP > 8)              Bandwidth-bound, not latency-bound

  PLACEMENT RULES SUMMARY:
    "Intra-layer" communication (TP, SP): MUST be on NVLink
    "Between-layer" communication (PP): IB adjacent-hop acceptable
    "Gradient sync" (DP): IB anywhere acceptable (overlapped with compute)
    "Expert routing" (EP): NVLink if EP ≤ 8; IB acceptable for larger EP
```

14.9 Key Takeaways and Review Questions

```
NETWORKING NUMBERS TO MEMORIZE COLD:

  NVLink 4.0 (H100, 8-GPU node):    900 GB/s bidirectional (all-to-all)
  NVLink 5.0 (B200, 8-GPU node):    1.8 TB/s bidirectional
  InfiniBand NDR per port:           50 GB/s bidirectional
  Per-node IB (8 NICs):              400 GB/s bidirectional
  RoCEv2 latency vs IB:             3-5 μs vs 1 μs (3-5× slower)

  Rail-optimized vs non-rail AllReduce: 85-90% vs 50-60% NIC utilization
  NCCL tuning gain:                  10-40% communication improvement
  NVLS gain vs Ring (intra-node):    ~2× for large messages
```

```
  IB SHARP gain vs standard:         50% less traffic, 30-40% lower latency
  GPUDirect RDMA CPU savings:        12 freed CPU cores per DGX node

  TP over IB vs NVLink (70B, 80L):   864ms vs 72ms per step (12× more expensive)
  Scaling efficiency (1,024 GPU):    74-85% target; <65% = investigate

KEY PRINCIPLES:

  1. TP MUST STAY WITHIN THE NVLINK DOMAIN — NO EXCEPTIONS:
     160 AllReduces per step × 5.4ms per AllReduce over IB = 864ms.
     The forward pass is ~400ms. TP over IB makes training impossible.
     Every NVLink violation costs more than a full extra forward pass.

  2. RAIL-OPTIMIZED TOPOLOGY IS THE HIGHEST-IMPACT NETWORKING DECISION:
     A correctly wired rail-optimized fabric with proper NCCL_IB_HCA pinning
     can double AllReduce throughput vs an unoptimized fabric.
     This is not software optimization — it is physical cabling and switch
     port assignment that must be planned before the cluster is deployed.

  3. OVERLAP IS THE HIGHEST-IMPACT SOFTWARE NETWORKING OPTIMIZATION:
     Unoptimized: step = compute + AllReduce (sequential).
     With overlap: step = max(compute, AllReduce).
     Going from 0% to 70% overlap at typical 30% AllReduce fraction:
     MFU improvement: +23% (relative). More valuable than most kernel
optimizations.

  4. NCCL TUNING YIELDS 10-40% IMPROVEMENT FROM ENVIRONMENT VARIABLES ALONE:
     NCCL_IB_HCA pinning + QPS_PER_CONNECTION=4 + NVLS/SHARP + BUFFSIZE.
     One hour of NCCL configuration work → permanent improvement for every run.

  5. IB AND ROCE HAVE DIFFERENT CONGESTION CONTROL REQUIREMENTS:
     IB: credit-based flow control built-in (PFC optional, SHARP available).
     RoCE: requires PFC (lossless) + DCQCN or CTS (active congestion control).
     Running RoCE without congestion control at 400G: incast → PFC storm →
collapse.

  6. MONITORING IB LINK SPEED IS NON-NEGOTIABLE:
     Degraded links (400G → 100G) are not always detected by standard monitoring.
     A single degraded link in the AllReduce ring slows the ENTIRE cluster.
     Add ibstatus bandwidth monitoring with alert at < 350 Gbps per port.
```

Review Questions

Difficulty: Foundational

Question 1 (TP over IB Overhead): Explain why Tensor Parallelism MUST stay within the NVLink domain. Calculate the exact overhead if TP=8 were run across InfiniBand NDR (50 GB/s effective per link) for a 70B model with 80 layers, assuming each AllReduce message is 268 MB and takes 5.4ms over IB per operation.

Expected answer: TP requires 2 AllReduces per transformer layer (one after attention, one after FFN). For 80 layers: 80 × 2 = 160 AllReduces per step. Over IB: 160 × 5.4ms = 864ms in communication overhead per step. The actual GPU forward/backward compute time is approximately 400ms forward + 900ms backward = 1,300ms. Total step time: 1,300ms + 864ms = 2,164ms, of which 40% is wasted on TP communication. Over NVLink (900 GB/s): 268 MB / (900/8) GB/s ≈ 0.45ms per AllReduce. 160 × 0.45ms = 72ms total. The 12× latency difference (5.4ms IB vs 0.45ms NVLink) makes TP over IB not just slower but architecturally impractical — it adds more time than the forward pass itself. TP over IB is not a performance tradeoff; it is a fundamental design failure.

Difficulty: Intermediate

Question 2 (NCCL Configuration for Scaling Improvement): A training cluster runs at 55% scaling efficiency at 1,024 GPUs. Nsight Systems shows AllReduce operations appearing *sequentially* after backward (not overlapped), consuming 28% of total step time. List three specific NCCL configuration or PyTorch changes, explain the mechanism of each, and estimate the MFU improvement.

Expected answer: (1) `BackwardPrefetch.BACKWARD_PRE` in FSDP or `gradient_as_bucket_view=True` + `bucket_cap_mb=25` in DDP: Enables NCCL AllReduce to launch on separate CUDA stream as soon as each gradient bucket fills, while the backward pass continues on the compute stream. Mechanism: buckets are registered as backward hooks; when bucket fills it triggers async NCCL launch. Expected gain: 15-20% of AllReduce can be overlapped immediately with remaining backward layers. If 28% of step is AllReduce and 70% overlaps: net overhead drops from 28% to 8.4%. Step time improves ~17%. (2) `NCCL_IB_HCA=mlx5_0:1,mlx5_1:1,...` pinned to rail topology: if NICs are not pinned, NCCL may use cross-rail paths for gradient communication, creating spine hotspots. Realigning NCCL to the rail topology can improve AllReduce bandwidth by 30-50%, which means AllReduce finishes faster and overlaps more fully. Mechanism: each GPU's NCCL uses only its dedicated rail NIC, preventing cross-rail interference. Expected gain: busbw increases from ~28 GB/s to ~43 GB/s; AllReduce duration drops 35%. (3) `NCCL_MIN_NCHANNELS=4` + `NCCL_IB_QPS_PER_CONNECTION=4`: Increases the number of RDMA queue pairs per NIC connection, allowing the 400G NIC ASIC to use more parallel DMA engines. A single QP on a 400G NIC often maxes at ~200 Gbps (single-threaded DMA). With 4 QPs: approaches full 400G line rate. Mechanism: more QPs = more parallel NIC threads = higher achieved bandwidth. Expected gain: 10-15% additional AllReduce throughput. Combined with (1) and (2): AllReduce

that was 28% of step time and unoptimized could become 6% of step time with 70% overlap, improving MFU from 55% to approximately 67%.

Difficulty: Principal Level

Question 3 (IB SHARP vs NVLS): Compare IB SHARP and NVLS. What problem does each solve, which network technology does each require, and where in the physical network hierarchy does each perform the AllReduce computation?

Expected answer: Both SHARP and NVLS solve the same problem: the overhead of running AllReduce summation on GPU Tensor Cores, consuming SM cycles and HBM bandwidth that could be used for model computation. Both offload the reduction computation to dedicated hardware. IB SHARP: performs AllReduce computation inside InfiniBand switch ASICs (Quantum2+). Requires InfiniBand fabric specifically — IB SHARP is NOT available on RoCE (no compute units in Ethernet switch ASICs). Operates at the inter-node level (Level 3 in the hierarchy). Reduction happens as packets flow through the switch fabric. Traffic reduction: ~50% vs standard Ring AllReduce (no need to send full gradients back). Hardware: SHARP compute units consume dedicated ASIC resources in Quantum2 switches. Works across multiple switch hops for large clusters. NVLS (NVLink SHARP): performs AllReduce computation inside NVSwitch chips on the DGX H100 node backplane. Requires NVLink fabric — only works intra-node (Level 2 in the hierarchy). Cannot operate across InfiniBand. The NVSwitch accepts gradients from all 8 local GPUs, sums them in dedicated aggregation compute units, and multicasts the result to all 8 GPUs simultaneously. Speedup: ~2× vs Ring AllReduce intra-node. Key distinction: NVLS is faster for intra-node (sub-microsecond with NVSwitch hardware), IB SHARP covers inter-node AllReduce but with higher latency. In practice on a multi-node H100 cluster: NVLS handles the intra-node component of hierarchical AllReduce, IB SHARP handles the inter-node component. Together they can reduce total AllReduce time by ~60% vs software-only Ring AllReduce.

Difficulty: Principal Level

Question 4 (4D Parallelism Network Mapping): A 4D parallel training job uses TP=8, PP=4, CP=2, DP=16 for 1,024 GPUs. Map each parallelism dimension to the physical network fabric it should use (NVLink vs IB), justify each choice, and describe what would happen to training speed if TP=8 were placed across IB instead of NVLink.

Expected answer: Total GPUs: 8 × 4 × 2 × 16 = 1,024 ✓. TP=8 → NVLink (mandatory). 8 GPUs within a single DGX/HGX node. TP AllReduces happen 160× per step, each 268 MB. Must use 900 GB/s NVLink (0.45ms each = 72ms total). IB would require 5.4ms × 160 = 864ms — exceeding the forward pass duration. PP=4 → InfiniBand, adjacent nodes. One activation tensor per microbatch flows between adjacent pipeline stages. Volume: micro_B × seq_len × d_model ×

2B per P2P send. At batch=2, seq=4096, d=8192: 2 × 4096 × 8192 × 2B = 134 MB per PP boundary. Over IB at 50 GB/s: 2.7ms per P2P send — acceptable as pipeline stages are sequential anyway. CP=2 → NVLink if two GPUs are on same node; IB if across nodes. Context parallelism uses All-to-All for attention K,V tensors. With CP=2, the two CP GPUs should ideally be co-located (same node) to keep on NVLink. If CP crosses a node boundary, the All-to-All adds ~2-3ms per transformer layer → degraded performance for long sequences. DP=16 → InfiniBand, any topology. Gradient AllReduce happens once per step, large message (140 GB × TP-sharded fraction). The 16 DP replicas can span any topology. ReduceScatter + AllGather is overlapped with compute (FSDP). IB handles the bandwidth requirement. If TP=8 were placed across IB: TP AllReduce: 160 steps × 5.4ms = 864ms per training step (IB latency). Total step time: forward (400ms) + backward (900ms) + TP-over-IB (864ms) = 2,164ms. Compare to correct config: 400 + 900 + 72ms (NVLink TP) = 1,372ms. Degradation: 2,164 / 1,372 = 1.58× slower. MFU drops proportionally: if baseline is 55% MFU, TP-over-IB gives 55% / 1.58 = 35% MFU. The training run would take 58% longer and cost 58% more compute budget for the same number of training steps.

Further Reading

1. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM* — Narayanan et al. (2021). `arxiv.org/abs/2104.04473` Section 4 covers the interplay between TP, PP, DP and the interconnect requirements. The AllReduce overlap analysis is the clearest published treatment.
2. *Reducing Communication in Distributed Training with SHARP* — Bloch et al. (2023). NVIDIA whitepaper. The SHARP aggregation tree protocol and performance measurements. `developer.nvidia.com/networking/sharp`
3. *Meta's RoCE-based Large-Scale AI Training* — Ryabinin et al. (SIGCOMM 2024). The engineering decisions behind the 24,576-GPU Llama-3 training cluster on RoCEv2. Best public treatment of CTS congestion control and incast avoidance at scale.
4. *NCCL Documentation and Source Code* — NVIDIA. `github.com/NVIDIA/nccl` Read: `src/algorithms/ring.cc` (Ring AllReduce implementation) and `src/transport/net_ib.cc` (IB transport layer). Understanding the source makes environment variable tuning much more intuitive.
5. *GPUDirect RDMA Technology Overview* — NVIDIA (2023). `docs.nvidia.com/cuda/gpudirect-rdma/` The technical reference for P2P memory registration and the DMA path.
6. *RoCEv2 Deployment Guide* — NVIDIA/Mellanox Application Note. The definitive reference for PFC configuration, DCQCN tuning, and QoS class separation for lossless

RDMA over Ethernet.

— End of Chapter 14 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved